Module 08: Planning — The Complete Active Inference Agent

Course: Active Inference 101 | **Unit**: Implementation | **Audience**: First-semester undergraduates

Learning Objectives

1. Implement **multi-step planning** with deep temporal policies.
2. Code **habit formation** via policy prior sharpening.
3. Build the **complete Active Inference agent** integrating all previous modules.

Key Concepts

1. The Complete Agent

```
 import numpy as np
from itertools import product

class CompleteActiveInferenceAgent:
    """
    Full Active Inference agent with:
    - Perception (variational inference)
    - Action (EFE-based policy selection)
    - Learning (Dirichlet parameter updates)
    - Habit formation (policy prior sharpening)
    """

    def __init__(self, A_prior, B_prior, C, D, policy_len=2, gamma=1.0):
        # Concentration parameters for learning
        self.a = A_prior.astype(float).copy()
        self.b = B_prior.astype(float).copy()
        self.C = C.copy()
        self.D = D.copy()
        self.gamma = gamma
        self.policy_len = policy_len

        # Point estimates
        self.A = self._dir_mean(self.a)
        self.B = self._dir_mean_3d(self.b)

        # State beliefs
        self.qs = D.copy()

        # Policy prior (habits) — starts uniform
        self.num_actions = self.B.shape[2]
        self.policies = np.array(list(
```

```
            product(range(self.num_actions), repeat=policy_len)))
        self.policy_prior = np.ones(len(self.policies)) / len(self.polici
        self.habit_strength = np.ones(len(self.policies))

        # Complete logging
        self.log = {
            'beliefs': [], 'actions': [], 'observations': [],
            'G': [], 'pragmatic': [], 'epistemic': [],
            'policy_probs': [], 'policy_prior': [],
            'learning_rate': [], 'free_energy': [],
            'habit_strength': []
        }

    @staticmethod
    def _dir_mean(alpha):
        return alpha / alpha.sum(axis=0, keepdims=True)

    @staticmethod
    def _dir_mean_3d(alpha):
        result = np.zeros_like(alpha)
        for a in range(alpha.shape[2]):
            result[:,:,a] = alpha[:,:,a] / alpha[:,:,a].sum(axis=0, keepd
        return result

    # --- PERCEPTION ---
    def infer_states(self, observation):
        """Bayesian belief update."""
        likelihood = self.A[observation, :]
        self.qs = likelihood * self.qs
        self.qs /= self.qs.sum()
        return self.qs
```

```
    # --- ACTION ---
    def evaluate_policies(self):
        """Compute EFE for all policies."""
        G = np.zeros(len(self.policies))
        prag = np.zeros(len(self.policies))
        epist = np.zeros(len(self.policies))

        for i, policy in enumerate(self.policies):
            G[i], prag[i], epist[i] = self._compute_efe(policy)

        return G, prag, epist

    def _compute_efe(self, policy):
        """Expected Free Energy with decomposition."""
        pragmatic_total = 0.0
        epistemic_total = 0.0
        qs_pred = self.qs.copy()

        for action in policy:
            qs_pred = self.B[:, :, action] @ qs_pred
            qo_pred = self.A @ qs_pred

            pragmatic_total += -(qo_pred * self.C).sum()

            epist = 0.0
            for s in range(len(qs_pred)):
                if qs_pred[s] > 1e-16:
                    po_s = self.A[:, s]
                    epist += qs_pred[s] * (
                        po_s * (np.log(po_s + 1e-16) - np.log(qo_pred + 1
                    ).sum()
            epistemic_total += epist
```

```
        return pragmatic_total - epistemic_total, pragmatic_total, episte

    def select_action(self):
        """Select action with habit-modulated policy selection."""
        G, prag, epist = self.evaluate_policies()

        # Combine EFE with policy prior (habits)
        log_pi = -self.gamma * G + np.log(self.policy_prior + 1e-16)
        pi = np.exp(log_pi - log_pi.max())
        pi /= pi.sum()

        chosen = np.random.choice(len(self.policies), p=pi)
        action = int(self.policies[chosen][0])

        # Log
        self.log['G'].append(G.copy())
        self.log['pragmatic'].append(prag.copy())
        self.log['epistemic'].append(epist.copy())
        self.log['policy_probs'].append(pi.copy())
        self.log['policy_prior'].append(self.policy_prior.copy())
        self.log['actions'].append(action)

        return action, chosen

    # --- LEARNING ---
    def learn(self, observation, action, prev_qs):
        """Update model parameters via Dirichlet updates."""
        obs_vec = np.zeros(self.a.shape[0])
        obs_vec[observation] = 1.0

        self.a += np.outer(obs_vec, self.qs)
```

```
        self.A = self._dir_mean(self.a)

        self.b[:, :, action] += np.outer(self.qs, prev_qs)
        self.B = self._dir_mean_3d(self.b)

        lr = 1.0 / self.a.sum()
        self.log['learning_rate'].append(lr)

    # --- HABIT FORMATION ---
    def update_habits(self, chosen_policy_idx, reward_signal=1.0):
        """Strengthen the habit for successful policies."""
        self.habit_strength[chosen_policy_idx] += reward_signal
        self.policy_prior = self.habit_strength / self.habit_strength.sum
        self.log['habit_strength'].append(self.habit_strength.copy())

    # --- COMPLETE STEP ---
    def step(self, observation):
        """One full step: perceive → decide → learn → habituate."""
        prev_qs = self.qs.copy()

        # Perceive
        self.infer_states(observation)
        self.log['beliefs'].append(self.qs.copy())
        self.log['observations'].append(observation)

        # Decide
        action, chosen_idx = self.select_action()

        # Learn
        self.learn(observation, action, prev_qs)

        # Habituate
```

```
        self.update_habits(chosen_idx)

        return action

    # --- REPORTING ---
    def summary(self):
        """Print agent summary."""
        n = len(self.log['actions'])
        print(f"=== Agent Summary ({n} steps) ===")
        print(f"Actions: {np.bincount(self.log['actions'], minlength=self
        print(f"Final beliefs: {self.qs.round(3)}")
        print(f"Learning rate: {self.log['learning_rate'][-1]:.6f}" if se
        print(f"Strongest habit: policy {np.argmax(self.habit_strength)}"
        print(f"Habit entropy: {-(self.policy_prior * np.log(self.policy_
```

2. Full Simulation

```
def run_full_simulation(agent, env, num_steps=50):
    """Run the complete agent-environment loop."""
    obs = np.random.choice(agent.A.shape[0], p=env.A[:, env.state])

    for t in range(num_steps):
        action = agent.step(obs)
        obs = env.step(action)

        if t % 10 == 0:
            print(f"t={t}: obs={obs}, act={action}, "
                  f"beliefs={agent.qs.round(2)}, "
                  f"lr={agent.log['learning_rate'][-1]:.4f}")

    agent.summary()
    return agent
```

Summary

The Complete Active Inference Agent integrates perception (Bayesian belief updating), action (EFE-based policy selection), learning (Dirichlet parameter updates), and habit formation (policy prior sharpening) into one unified class. This is the implementation capstone of the 101 course — every equation from the Mathematical Frameworks unit and every concept from Cognitive Science and Computational Neuroscience is realized in working Python code.

Further Reading

- Heins, C. et al. (2022). pymdp: A Python library for active inference in discrete state spaces. *JOSS*, 7(73), 4098.
- Parr, T., Pezzulo, G., & Friston, K. J. (2022). *Active Inference*. MIT Press.

• Da Costa, L. et al. (2020). Active inference on discrete state-spaces. *Journal of Mathematical Psychology*, 99, 102447.